
Structured Evolution for Search-Space Generation in ML Kernel Scheduling

Kevin Chan
Stanford University
tsekchan@stanford.edu

Newton Chen
Stanford University
hsinchen@stanford.edu

Abstract

We study where a zero-shot large language model should intervene in an ML kernel scheduling pipeline, and whether its flexibility and expressiveness can outperform a rule-based autotuning baseline, Apache TVM MetaSchedule. Rather than asking an LLM only to emit one concrete low-level schedule, we compare that direct approach with a structured alternative in which the LLM generates a MetaSchedule design space and TVM searches within it. We build this comparison on fixed TensorIR matrix-multiplication kernels, automatic syntax/compile/correctness/latency gates, and an OpenEvolve-style mutation-and-selection loop over generated programs. Our results do not support the hypothesis that LLM-generated search spaces consistently outperform direct schedule generation or standard MetaSchedule: MetaSchedule remains strongest on regular matrix multiplication, direct schedule evolution is less reliable but finds large speedups on irregular shapes where MetaSchedule fails, and search-space evolution is more conservative and sensitive to prompt, seed, and feedback quality. These findings suggest that the main promise of LLM-guided scheduling is not replacing mature autotuners on their best cases, but expanding scheduling coverage for irregular or underspecified workloads.

Github: <https://github.com/t-sekai/structured-evolve>.

1 Background and setup

Machine learning systems often spend much of their time in tensor kernels such as matrix multiplication. The mathematical computation of a kernel is fixed, but its *schedule*—tiling, loop ordering, thread binding, memory hierarchy choices, and unrolling—can change latency by large factors. Modern autotuners such as Apache TVM MetaSchedule search over schedule spaces, compile candidates, test correctness, and benchmark latency (Shao et al., 2022; Apache TVM contributors, 2026). The central question in this project is whether a zero-shot LLM can improve this process, and especially whether it is better used to generate concrete schedules or to generate structured search spaces for an autotuner.

Inputs and outputs. The input to our system is a TensorIR kernel representing a fixed computation. The output is a valid optimized schedule, together with measured latency and speedup relative to a fixed baseline schedule. Invalid generated code, schedules that fail compilation, and schedules that produce incorrect outputs are rejected automatically before latency is used as fitness.

Goals and constraints. Our goal is to test where LLMs should intervene to improve kernel performance over the MetaSchedule baseline. The constraints are deliberately strict: the kernel computation cannot change; the LLM is used in a zero-shot setting; all candidates must pass automated syntax, compilation, correctness, and benchmarking gates; and comparisons use the same hardware, evaluator, and baselines. We therefore evaluate the LLM as a scheduling assistant rather than as a source of new algorithms.

Research question. Our motivating hypothesis is that zero-shot LLMs can use their flexibility and expressiveness to improve kernel scheduling beyond traditional rule-based autotuners, especially by proposing schedules or schedule spaces that are not easily captured by fixed hand-engineered rules. Within that broader hypothesis, our main falsifiable comparison is: *zero-shot LLMs improve kernel scheduling more effectively when they generate search spaces for an existing autotuner than when they directly generate one concrete schedule*. This hypothesis would be supported if the search-space approach consistently produced competitive or better speedups with higher validity and robustness. It would be weakened if direct schedule generation produced better speedups or if standard MetaSchedule dominated both LLM-guided methods; as our results show, the latter outcome is closer to what we observed on regular matrix multiplication.

Technical crux. The hard part is not simply calling an LLM; it is designing a representation and feedback loop that converts noisy generated code into useful scheduling progress. Schedules are brittle: small syntactic or structural errors can break compilation, and a schedule that is valid may still be slower than the default. The project therefore required us to build an evaluator that safely rejects invalid candidates, records feedback, ranks candidates by latency, and uses evolutionary selection to guide future mutations.

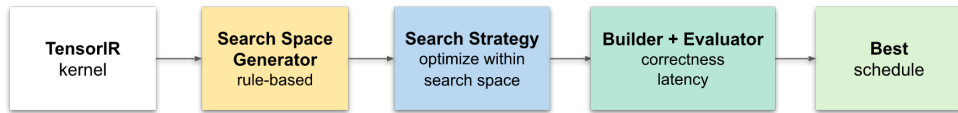
2 Approach

We built on Apache TVM MetaSchedule because it follows the Halide-inspired separation between the tensor algorithm and its schedule (Ragan-Kelley et al., 2013; Shao et al., 2022; Apache TVM contributors, 2026). This separation allows generated code to mutate scheduling decisions without changing the underlying computation. TVM also provides the infrastructure needed for fair experiments: design-space generation, search, compilation, correctness checking, and benchmarking.

Figure 1 shows the two pipeline components that we combine. The MetaSchedule baseline starts from the fixed TensorIR computation, generates and ranks candidate schedules, compiles and checks them, and reports the best measured latency. The evolutionary loop sits outside this evaluator: it prompts the LLM to mutate a candidate program, sends the resulting program through the same validity and benchmarking gates, and feeds the outcome back into survivor selection for the next generation.

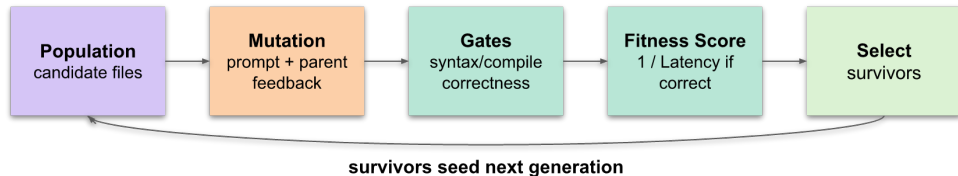
We compared two LLM intervention levels, summarized in Figure 2. **Level 1** asks the LLM to write one complete TVM schedule function, `apply_schedule(...)`. The evolutionary loop mutates

Apache TVM MetaSchedule



(a) TVM MetaSchedule baseline

OpenEvolve-Style Evolution



(b) Evolutionary mutation loop

Figure 1: System-level pipelines. MetaSchedule provides the rule-based autotuning baseline and the inner tuner used by Level 2, while the OpenEvolve-style loop wraps generated programs with mutation, evaluation, selection, and feedback.

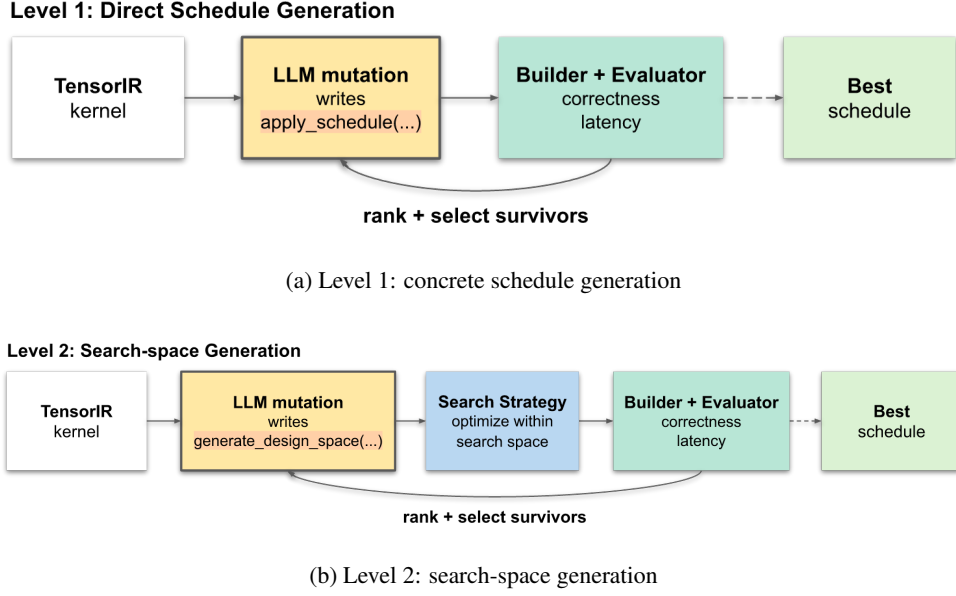


Figure 2: LLM intervention points. Level 1 directly evolves schedule functions that are compiled and benchmarked as concrete candidates. Level 2 evolves design-space generators, after which TVM searches within the generated space before correctness and latency are measured.

concrete schedule code, evaluates it, and selects the best surviving schedules. This tests whether direct code mutation can discover useful CUDA scheduling structure. **Level 2** asks the LLM to write a structured design-space generator, `generate_design_space(...)`. TVM then searches within the generated space and returns the best schedule it finds. This tests whether the LLM can choose useful families of schedules while relying on TVM for low-level tuning details.

Both levels use an OpenEvolve-style loop (Sharma, 2025). A population contains candidate files. Mutation prompts include a parent candidate and feedback from earlier evaluations. Each candidate passes through gates for syntax, compilation, and correctness. Correct candidates receive fitness proportional to inverse latency, so lower latency gives higher fitness. Survivors are carried forward to seed the next generation, and artifacts such as prompts, responses, candidate code, ranking, and tune history are stored to improve later mutations.

The default evolutionary configuration used GPT-OSS-120B for 10 generations with population size 8, 3 survivors, elite carry-forward disabled, diversity inspiration enabled, rejection cascade enabled, and evaluator feedback enabled. TVM search used an XGBoost cost model and gradient task scheduler, with search budgets of 128 global trials for design-space search and 1024 global trials for the main MetaSchedule tuning loop.

3 Evaluation and results

Definition of success. Success means producing correct schedules with lower latency than the fixed baseline, and ideally doing so more robustly than direct schedule generation. We measured three quantities. First, **latency** measures the wall-clock runtime of the compiled kernel after correctness has been established. Second, **speedup** is reported as baseline latency divided by candidate latency, so values above $1\times$ indicate an improvement. Third, **validity** measures whether an attempted run produced a candidate that passed all gates and could therefore be included in the latency comparison. The key comparison is among four methods: the fixed baseline schedule, default TVM MetaSchedule, Level 1 concrete schedule evolution, and Level 2 search-space evolution.

Experimental setup. We evaluated CUDA matrix-multiplication workloads, including regular square shapes and irregular shapes. All methods operate on the same fixed TensorIR computation, so a schedule is only allowed to change execution order, memory placement, loop structure, and

parallelization strategy. Each attempted candidate follows the same evaluator path: parse generated Python/TVM code, run syntax and API checks, compile the scheduled TensorIR, test numerical correctness against the reference computation, and benchmark only candidates that pass correctness. This path is important because it prevents invalid LLM output from being treated as a poor latency measurement; invalid candidates are counted as failures instead. The figures report speedup relative to the fixed baseline, so values above 1 indicate improvement. Error bars summarize variation across successful runs, and annotations such as “3/3” indicate the number of successful runs out of attempted runs.

Baselines and compared systems. The **fixed** baseline is the unoptimized or default hand-written schedule used to normalize all speedups and seed our LLM methods. **MetaSchedule** is TVM’s standard search-based autotuner with the same compute definition and evaluator. **Level 1** evolves complete schedule functions, so the LLM directly controls concrete primitives such as splitting loops, reordering loops, binding loops to CUDA blocks and threads, caching tensors, and adding unrolling or vectorization decisions when available. **Level 2** evolves a generator for a family of schedules; TVM then searches inside that generated family. This distinction matters for interpretation: Level 1 tests LLM schedule-writing ability, while Level 2 tests whether the LLM can expose useful schedule choices to an existing autotuner.

What counts as a failed run. A run can fail for several reasons: generated code may be syntactically invalid, may call TVM scheduling APIs incorrectly, may produce an illegal schedule that cannot be lowered or compiled, or may compile but fail the correctness check. These failures are expected in an LLM-based code-generation loop and are not hidden by the plots. Instead, the success-count labels show how many independent attempts produced reportable latency numbers. This is why the irregular workload result is especially meaningful: MetaSchedule reports 0/3 successful runs there, while the LLM-guided methods still produce valid schedules.

3.1 Main comparison

Figure 3 shows the primary results. On regular matrix multiplication, default TVM MetaSchedule is the strongest method: it reaches about $2.2\times$ speedup on the $256 \times 256 \times 256$ case and about $15\times$ on the $8 \times 256 \times 1024$ case. This result falsified our hypotheses that LLM can outperform MetaSchedule with its flexibility and expressiveness; however, it makes sense because regular matrix multiplication is exactly the kind of workload for which TVM’s hand-engineered search rules and cost model are mature.

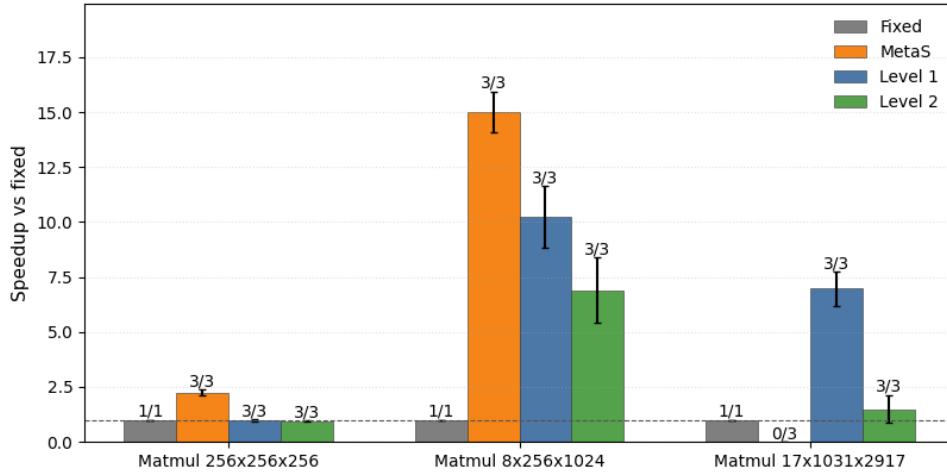


Figure 3: Main experiment speedups relative to the fixed baseline. Labels above bars show successful runs. MetaSchedule dominates regular matrix multiplication, while direct Level 1 evolution finds the largest gain on the irregular shape where MetaSchedule produced no successful result.

Level 1 direct schedule evolution is less reliable but can be powerful. On the $8 \times 256 \times 1024$ workload it reaches roughly $10\times$ speedup, below MetaSchedule but still far above the fixed baseline. On the irregular $17 \times 1031 \times 2917$ workload, MetaSchedule has no successful runs in our experiment, while Level 1 reaches about $7\times$ speedup. This is the clearest evidence that direct LLM-generated schedules can handle cases outside the most regular autotuning path.

Level 2 search-space generation is more conservative. It improves over the fixed baseline on the irregular workload and obtains substantial speedup on the $8 \times 256 \times 1024$ workload, but it does not match MetaSchedule on regular cases or Level 1 on the irregular case. This partially falsifies our initial hypothesis: generating search spaces did not dominate direct schedule generation in speedup. However, the Level 2 results still support the idea that structured LLM intervention can produce valid, tunable spaces, and the ablations below suggest that its quality depends strongly on feedback and prompting.

The shape dependence is the most important pattern in Figure 3. The square $256 \times 256 \times 256$ case leaves relatively little room for the LLM methods because MetaSchedule already has strong schedule templates for this workload. The skinny/batched $8 \times 256 \times 1024$ case has more opportunity for schedule specialization, so both LLM methods become competitive even though MetaSchedule still wins. The irregular $17 \times 1031 \times 2917$ case stresses assumptions built into standard schedule rules; here the direct Level 1 method is the only method with a large successful speedup. Thus, our results suggest that LLM-guided scheduling is most promising when the workload is outside the regular shapes for which standard autotuners are heavily optimized.

3.2 Hyperparameter sweep

Before the main experiments, we swept Level 1 evolutionary settings on CUDA matrix multiplication. The sweep varies two knobs: **population size**, the number of candidate schedules evaluated in each generation, and **survivors**, the number of candidates retained as parents for the next generation. Figure 4 shows that all tested settings produced valid candidates in all three runs, but latency varied from 0.0159 ms to 0.0257 ms. The best setting used population size 8 and 3 survivors. This supports using a moderately diverse population: too few candidates reduce exploration, too few survivors can discard useful partial schedule structure, and too many survivors can preserve weaker candidates and slow convergence. The result is empirical rather than conclusive, but it gave us a concrete setting for the main runs.

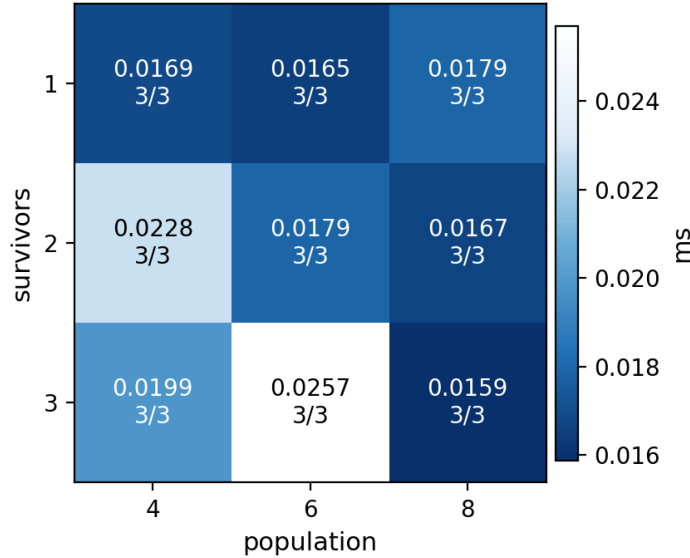


Figure 4: Level 1 hyperparameter sweep over population size and number of survivors. Each cell reports latency in milliseconds and successful runs. The best measured setting used population 8 and 3 survivors.

3.3 Evolution over generations

Figure 5 shows that the evolutionary loop improves performance over generations, but the two intervention levels improve in different ways. Level 1 starts from roughly 0.76 ms at the seed and rapidly drops to about 0.117 ms by generation 4, eventually reaching about 0.111 ms by generation 9. Its candidate stream is noisier, with valid candidates varying around 4/8 to 6.7/8, but selection can exploit rare high-quality concrete schedules once they appear. Level 2 is more consistently valid, often staying near 7/8 correct candidates per generation, but its latency improves more slowly and reaches only about 0.50 ms by generation 10. This reinforces the main comparison: direct schedule evolution trades reliability for larger speed gains, while structured search-space evolution is safer but less aggressive on this irregular matmul.

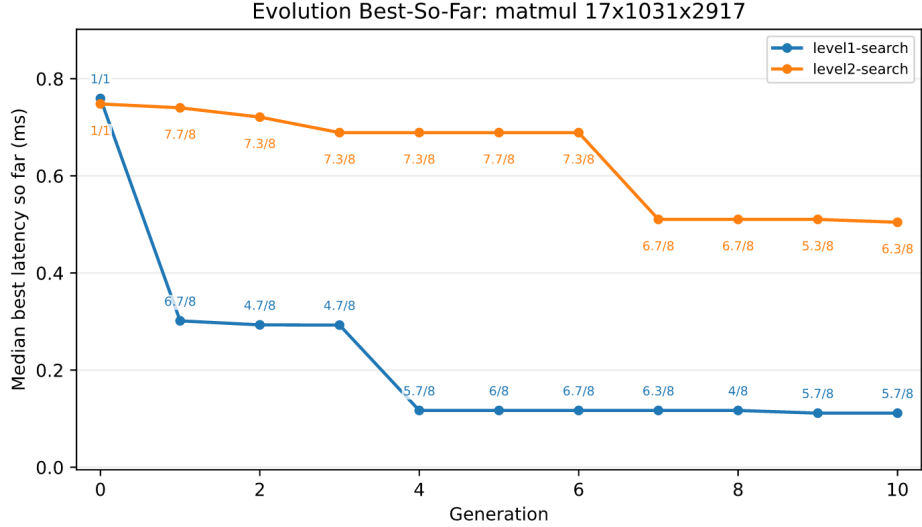


Figure 5: Evolution dynamics on the irregular $17 \times 1031 \times 2917$ matrix multiplication workload. The y-axis shows median best latency so far across three seeds, so lower is better. Labels above each point report the average number of valid candidates in that generation; generation 0 is the single seed candidate.

3.4 Ablations

The ablation experiment isolates which pieces of the evolutionary loop matter. **Default** is the reference configuration used for normalization in this plot. **+elite** enables elite carry-forward, meaning the best candidate from a generation is preserved directly rather than relying only on mutated descendants. **No diverse** disables diversity inspiration in survivor selection: instead of carrying one non-top but valid candidate into the next generation to preserve exploration, the loop selects survivors only by correctness and latency. **No reject** disables the rejection cascade, allowing more weak or invalid candidates to proceed farther through the loop before selection removes them. **No feedback** removes evaluator feedback from mutation prompts, so the LLM does not see concrete information about syntax errors, compile errors, correctness failures, or latency rankings from previous candidates. All ablation bars are normalized to the default configuration for the same level, making values above 1 better than default and values below 1 worse than default.

Figure 6 studies feature knobs in the evolutionary loop. We focus on ablations whose error bars are visually separated from the default, since those provide the clearest evidence of a real behavior change. In Level 1, **no feedback** is clearly worse than default. This makes sense because Level 1 asks the LLM to write a complete concrete schedule: without syntax, compile, correctness, and latency feedback, the mutation step has little information about which low-level TVM scheduling choices were invalid or slow. The result is more drift away from working schedule structure. **No diverse** is also below default for Level 1, suggesting that direct schedule evolution benefits from carrying at least one non-top valid survivor forward. Concrete schedules can be brittle and locally

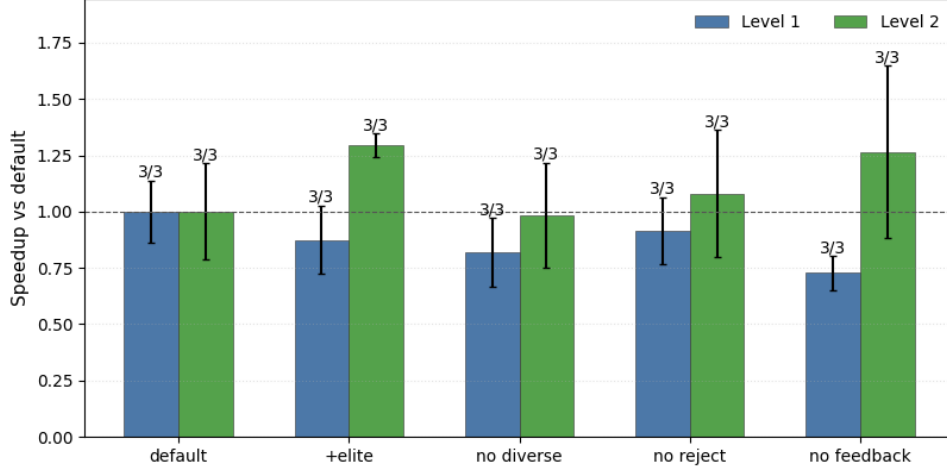


Figure 6: Ablation of evolutionary features. Values are normalized speedup versus the default configuration for each level. Level 1 generally degrades when feedback, diversity, or filtering are removed; Level 2 shows a different pattern, with some variants improving but with substantial variance.

specialized, so selecting only the current top correctness/latency candidates can prematurely collapse the population around a narrow schedule pattern, while a diverse survivor can preserve alternative loop transformations or memory placements that later mutations improve.

The **+elite** and **no diverse** settings show clearer effects for Level 2. **+elite** improves over the Level 2 default, suggesting that search-space generation benefits from preserving a strong incumbent design-space generator across generations. Unlike Level 1, where a single concrete schedule can over-specialize, Level 2 candidates define families of schedules that TVM can still explore internally; carrying forward a good family therefore gives the inner tuner a stable base to refine. **No diverse** is closer to default and does not show the same strong separation. This suggests that adding a non-top survivor for diversity is not always decisive for Level 2: the extra survivor can preserve an alternative design-space family, but TVM’s inner search may already provide enough exploration, and selecting only the top correctness/latency candidates can be similarly effective under this budget.

No reject produces results similar to the default, and this is also reasonable. The rejection cascade is mainly an efficiency and hygiene mechanism: it prevents obviously broken candidates from consuming later evaluator effort. However, the final plotted speedup is computed only after candidates pass correctness and benchmarking. Therefore, disabling early rejection can waste more evaluations or allow more bad candidates into intermediate stages, but the surviving candidates that reach the final benchmark are still filtered by the same compile/correctness/latency gates. Similar final speedups indicate that the downstream evaluator and survivor selection are strong enough to recover, even if early rejection helps keep the loop cleaner.

The broad conclusion is that the evolutionary controls have level-dependent effects: direct schedule evolution needs feedback to repair brittle concrete code, while search-space evolution benefits more from preserving good design-space structure and is less sensitive to whether survivor selection is purely top-ranked or includes one exploratory non-top candidate.

3.5 Prompt and seed improvements

After the main comparison, we investigated whether LLM evolution was stuck outside the CUDA schedule family that MetaSchedule was able to find. The original prompts accidentally mixed LLVM and CUDA constraints and asked the LLM to prefer simple, valid schedules. This made the generated schedules tend toward simple split, bind, and unroll patterns rather than more complete CUDA matmul structure such as local accumulation, shared-memory staging of operands, cooperative or vectorized loads, and thread/block tiling. We therefore tested three cumulative improvements in Figure 7: **+ better prompting** cleans up the target-specific prompt and explicitly asks for CUDA hierarchy; **++ stronger seed** starts from a more ambitious CUDA schedule with local accumulation

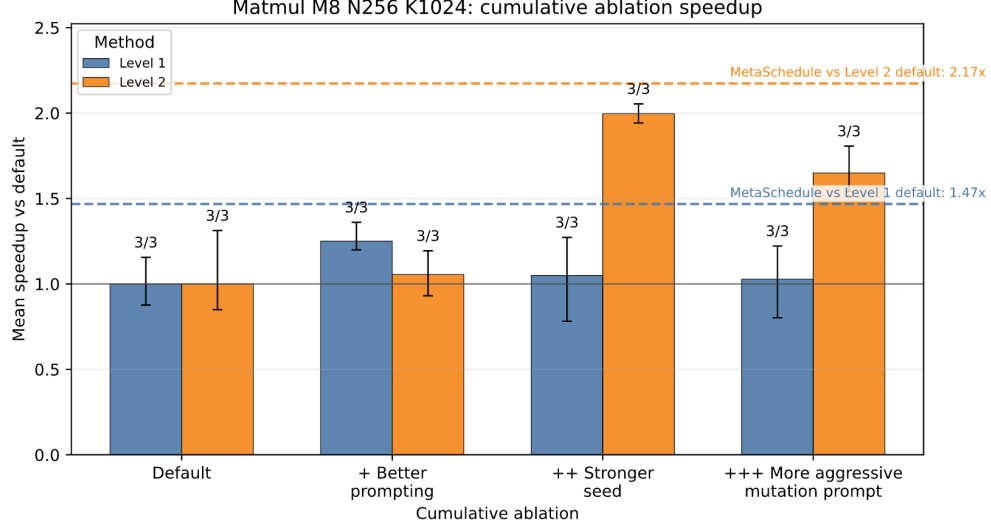


Figure 7: Follow-up prompt and seed improvements on the $8 \times 256 \times 1024$ matrix multiplication workload. Bars show mean speedup over the old default configuration for each level after CUDA-specific prompt cleanup, a stronger CUDA seed, and a more aggressive mutation prompt.

and shared-memory staging; and **+++ more aggressive mutation prompt** asks the model to make larger legal structural CUDA changes rather than only retuning constants in the parent.

The effects are level-dependent. For Level 1, better prompting is the main improvement, giving roughly $1.25\times$ mean speedup over the old default. Adding the stronger seed does not improve the mean as much, suggesting that direct schedule evolution can use clearer CUDA instructions but does not automatically preserve or refine an ambitious seed under freeform mutations. For Level 2, the stronger seed matters more: better prompting alone gives only a small gain, but the ambitious seed and more aggressive mutation prompt produce much larger speedups. This matches the intuition behind search-space generation: once the LLM exposes a richer CUDA schedule family, TVM can tune within that family instead of relying on the LLM to maintain one fragile concrete schedule.

3.6 Interpretation

The results do not show a simple victory for one method. MetaSchedule is best on regular matrix multiplication because its search rules are specialized and its cost model has a strong inductive bias for these workloads. Level 1 is brittle but expressive: it can make large, direct scheduling changes and therefore sometimes succeeds where the standard search path fails. Level 2 is safer conceptually, but under our limited budget it may generate spaces that are too broad, too narrow, or missing the specific schedule family needed for peak performance. The follow-up prompt and seed results refine this interpretation: some of the gap came from interface design rather than only from model capability, because clearer CUDA-specific prompts and stronger seeds improved both levels.

The likely speedup limits differ by method. For MetaSchedule, the limitation on irregular shapes appears to be search-space coverage or robustness rather than raw optimization once a valid schedule family is available. For Level 1, the limitation is validity and local code quality: the LLM can emit strong schedules, but many mutations are syntactically invalid, semantically invalid, or simply slower. Better CUDA-specific prompting helps, but a strong concrete seed is not automatically stable under mutation. For Level 2, the limitation is a two-stage credit-assignment problem. A slow final schedule could mean the LLM proposed the wrong design space, TVM searched the right space insufficiently, or the cost model ranked candidates poorly. The stronger-seed experiment suggests that Level 2 benefits when the generated space already contains richer CUDA structure, because TVM can then perform the low-level tuning within that family. More trials, better prompts, and richer feedback that explains which schedule rules mattered would likely help separate these causes.

Overall, our initial hypothesis is only partially supported. Structured search-space generation is a promising interface because it preserves TVM’s strengths and provides a safer abstraction than raw schedule code. However, in our experiments it did not consistently beat direct schedule evolution, especially on the irregular matmul where Level 1 found rare but much faster schedules. The strongest takeaway is that the right LLM intervention point is workload and interface dependent: mature autotuners remain hard to beat on regular workloads, direct LLM scheduling can pay off when rare high-quality schedules are selectable, and search-space generation becomes more attractive when prompts and seeds expose the right CUDA schedule family for TVM to tune.

4 Team responsibilities

Kevin Chan and Newton Chen jointly designed the project, implemented the TVM/MetaSchedule evaluation workflow, developed the OpenEvolve-style mutation and selection loop, ran the experiments, prepared the presentation, and analyzed the results. Kevin focused primarily on the Level 2 search-space generation experiments and ablations. Newton focused primarily on the Level 1 direct schedule generation experiments and hyperparameter sweep. Both team members contributed to debugging, result interpretation, and the final writeup.

5 References

References

- Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 519–530, 2013.
- Junru Shao, Xiyu Zhou, Siyuan Feng, Bohan Hou, Ruihang Lai, Hongyi Jin, Wuwei Lin, Masahiro Masuda, Cody Hao Yu, and Tianqi Chen. Tensor program optimization with probabilistic programs. *Advances in Neural Information Processing Systems*, 35:35783–35796, 2022.
- Apache TVM contributors. TVM MetaSchedule documentation. https://tvm.apache.org/docs/reference/api/python/meta_schedule.html.
- Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. GitHub, 2025. <https://github.com/algorithmicsuperintelligence/openevolve>.